Supervised Learning Classification: DECISION TREE Pruning

Course instructor: Prasanna Ranjith Christodoss
Associate Professor of Comp. Sc., Messiah University, PA 17055

He cuts off every branch in me that bears no fruit, while every branch that does bear fruit he prunes so that it will be even more fruitful.
John 15: 2-3

Outline

- Decision Tree as a classifier
- Pruning
- Build DT with Python

Pruning

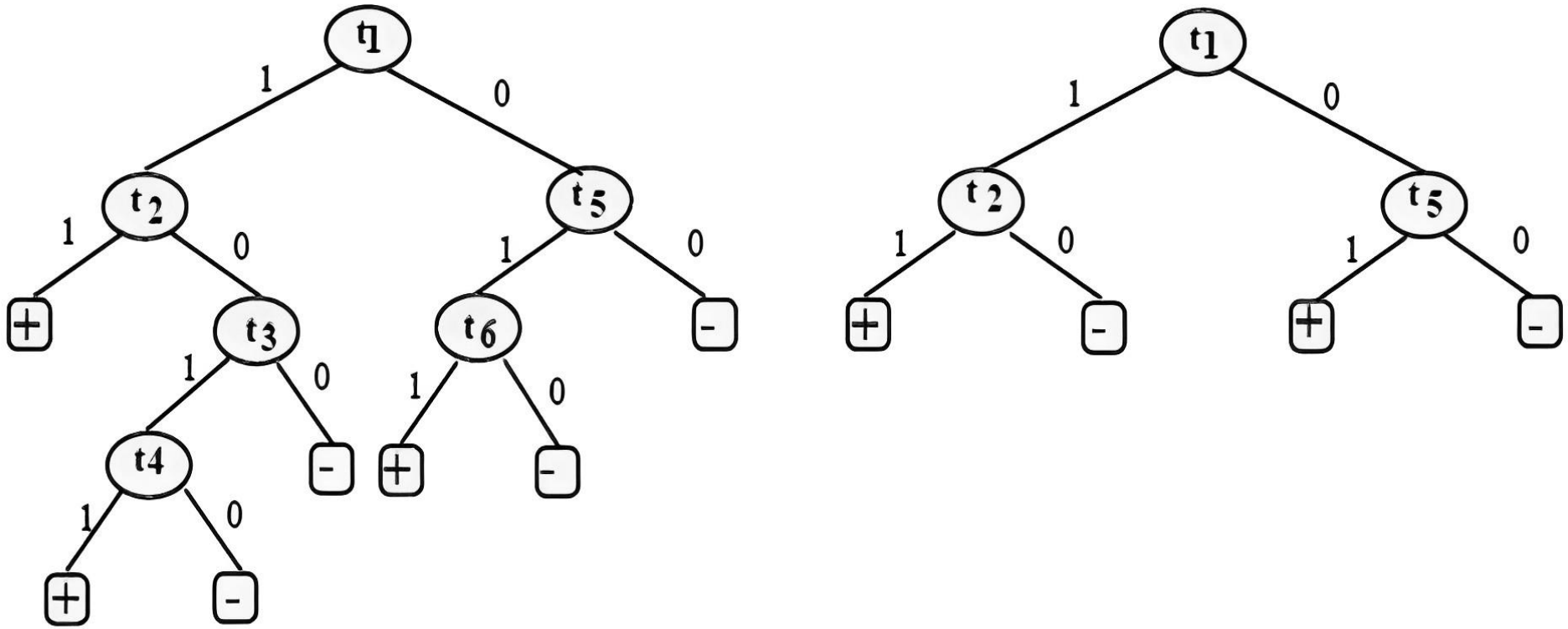
There are several advantage for building small decision trees in ML. These advantages include:

- **Interpretability:** Small decision trees are easier to understand and explain because they have fewer branches and nodes, making it simpler to follow the decision-making process.
- **Removal of Irrelevant and Redundant Attributes:** Small decision trees tend to focus on the most important attributes, filtering out irrelevant or redundant ones. This helps in reducing noise and improving the tree's predictive accuracy.
- **Lower Danger of Overfitting:** Overfitting occurs when a decision tree is too complex and captures noise in the data, leading to poor generalization to new, unseen data. Small decision trees are less prone to overfitting because they are simpler and capture the underlying patterns more effectively.

These advantages were the driving factors behind the use of information gain in decision tree induction. Information gain helps identify which attributes are most valuable for making decisions, aligning with the virtues mentioned above. Additionally, these advantages also provide a rationale for the next step in the process: pruning. Pruning involves trimming unnecessary branches from the tree to further enhance its simplicity and generalization capabilities.

wisdom expertise

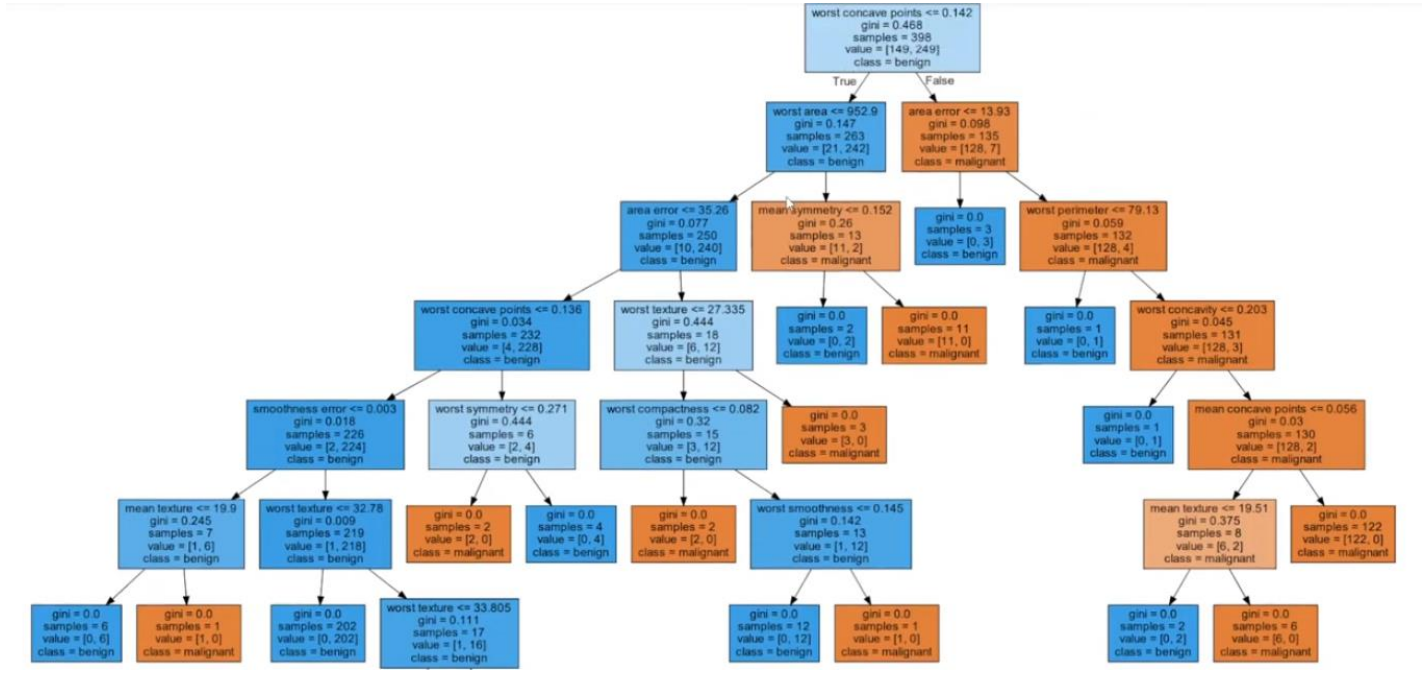
Pruning



On the left is the original decision tree with six tests: t1,...t6. On the right is its pruned version

How do we avoid overfitting

- Pre-pruning
- Post-pruning



Overfitting

Test data not seen by model during training

setosa	5.7 cm	4.4 cm	1.5 cm	0.4 cm
setosa	5.4 cm	3.9 cm	1.3 cm	0.4 cm
setosa	5.1 cm	3.5 cm	1.4 cm	0.3 cm
setosa	5.7 cm	3.8 cm	1.7 cm	0.3 cm
setosa	5.1 cm	3.8 cm	1.5 cm	0.3 cm

5.1 cm	3.8 cm	1.5 cm	0.3 cm

Output: Virginica

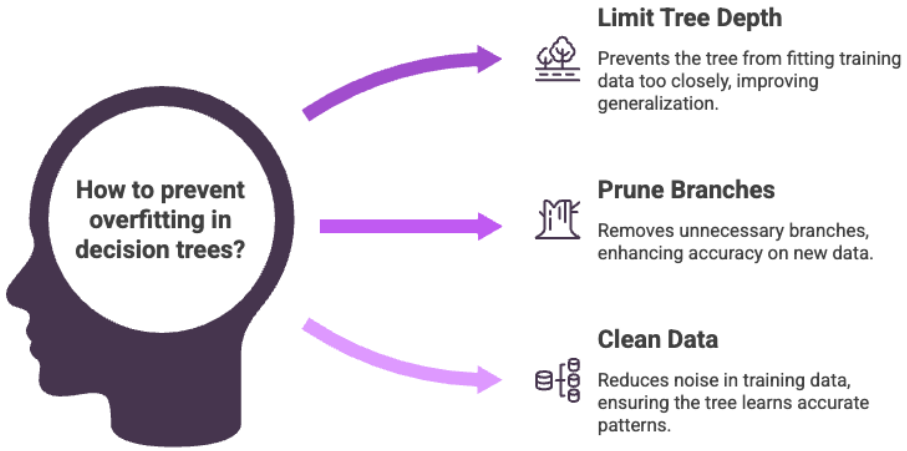
Overfitting

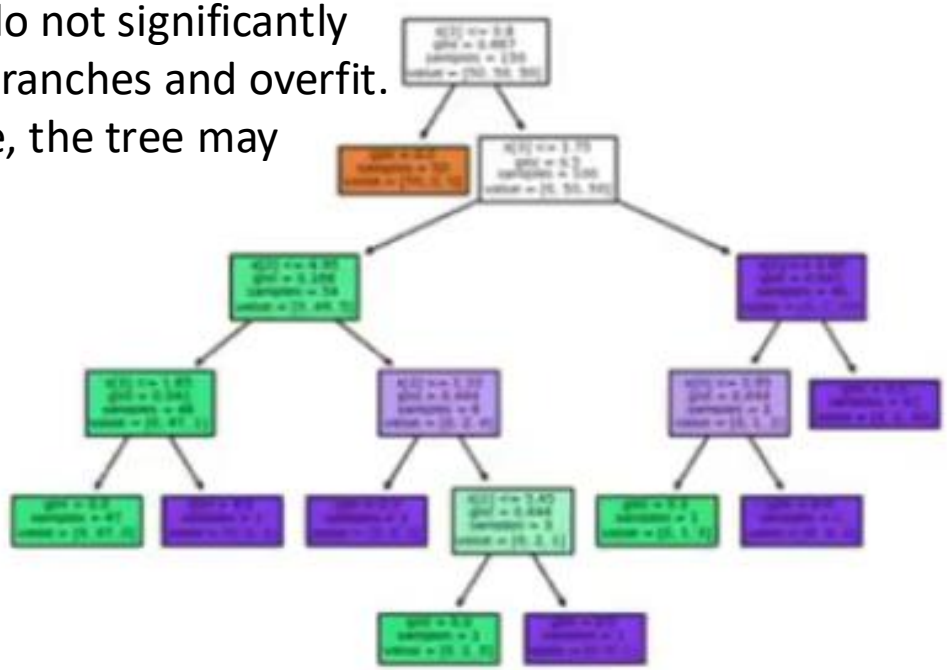
Overfitting occurs when a model fits the training data too closely, losing its ability to generalize to new information.

•**The tree is too deep:** A tree with many levels and branches may fit training data perfectly but fail on new data.

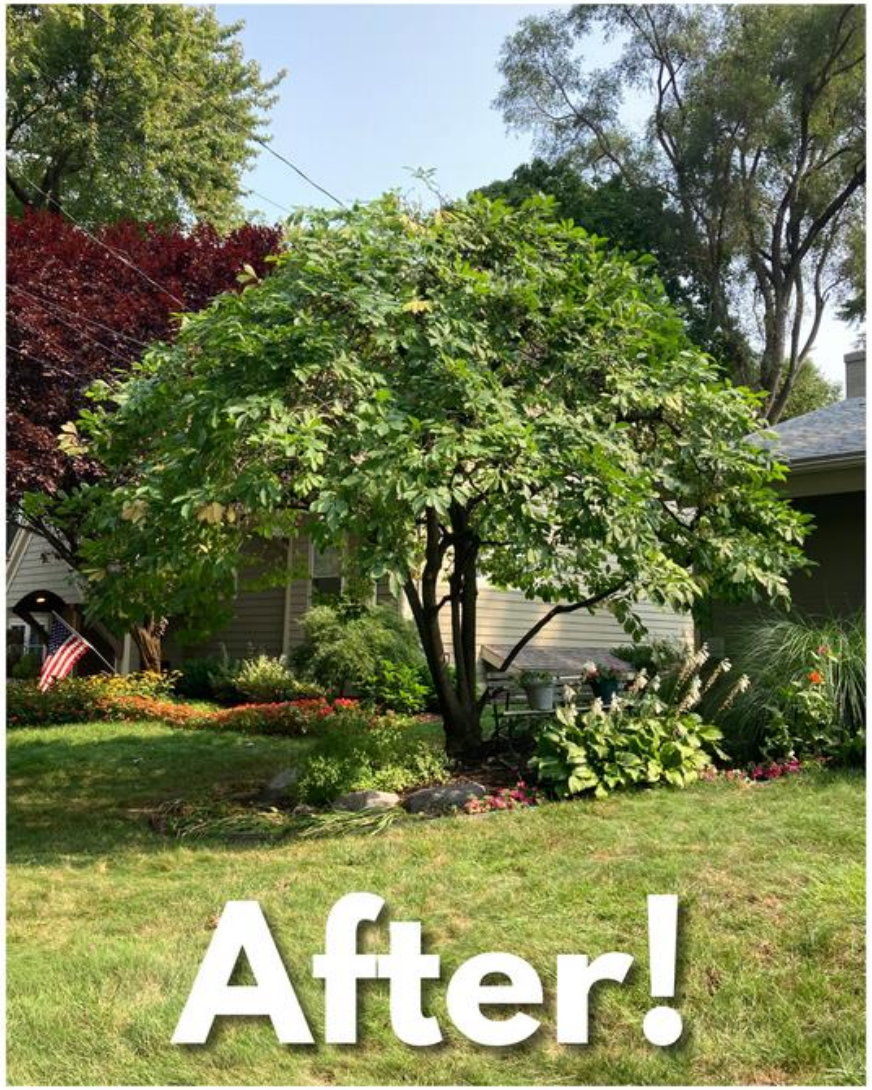
•**The tree is not pruned:** Pruning removes branches that do not significantly improve accuracy; without it, a tree may have too many branches and overfit.

•**The training data is noisy:** If data contains errors or noise, the tree may incorporate that noise into its logic, leading to overfitting.





Post-Pruning



Pre-Pruning

Pre-pruning is a technique used to stop a tree from growing beyond a certain size to prevent overfitting.

Pre-Pruning

Pre-pruning Techniques

Pre-pruning is a technique used to stop a tree from growing beyond a certain size to prevent overfitting. It can be achieved by:

1. Setting a **maximum depth** for the tree.
2. Setting a **minimum number of samples required at a leaf node**.
3. Setting a **maximum number of leaf nodes**.

While pre-pruning reduces overfitting risk, it may lead to a less complex tree with lower accuracy.

Pre-Pruning

1.Setting a **maximum depth** for the tree. (eg. depth=5)

Pre-Pruning

2. Setting a **minimum number of samples required at a leaf node**.

Pre-Pruning

2. Setting a **minimum number of samples required at a leaf node**.
(eg. No of Samples=3)

ID	Temperature	Play
1	Hot	No
2	Hot	No
3	Hot	No
4	Mild	Yes
5	Mild	Yes
6	Mild	Yes
7	Cool	Yes
8	Cool	Yes
9	Cool	No

```
      Temperature
     /     |     \
   Hot   Mild   Cool
   No    Yes   Yes / No
                (2) (1)
```

Pre-pruning rule:
*A leaf node must contain **at least 3 samples***

```
      Temperature
     /     |     \
   Hot   Mild   Cool
   No    Yes    Yes
```

Pre-Pruning

3. Setting a **maximum number of leaf nodes**. (LN=3)

```
            Income
        /       |       \
     Low    Medium    High
       |        |        |
   Credit   Credit   Credit
   Score    Score    Score
   /   \    /   \    /   \
 No    No  No   Yes Yes   Yes
```

Leaf nodes (exactly 3)

- Income = Low → No
- Income = Medium → No
- Income = High → Yes

As the tree grows:

1. It creates **leaf #1** ✅
2. Leaf #2 ✅
3. Leaf #3 ✅
4. Attempt to create **leaf #4** ❌ → **BLOCKED**

Further splits are not allowed, even if they reduce impurity.

Pre-Pruning

1. Setting a **maximum depth** for the tree.
2. Setting a **minimum number of samples required at a leaf node**.
3. Setting a **maximum number of leaf nodes**.

Pre-pruning can reduce the risk of overfitting

But it may also result in a less complex tree with lower accuracy

Pre-Pruning

1. Setting a **maximum depth** for the tree.
2. Setting a **minimum number of samples required at a leaf node**.
3. Setting a **maximum number of leaf nodes**.

How do you determine the values/limits for pre-pruning?

- ✓ **CV**
- ✓ **GridSearchCV**

```python
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn import tree
from sklearn.metrics import accuracy_score

#Visualization of Decision Tree
import graphviz
```

E1

```python
#Load Data
cancer_Data = load_breast_cancer(as_frame=True)

X=cancer_Data["data"]
y=cancer_Data["target"]

#retrieve feature name and class label for visualization
feature_names = cancer_Data["feature_names"]
target_names=cancer_Data["target_names"]

X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.30, random_state=42)

#DT Model
clf= tree.DecisionTreeClassifier()
#Training the Model
clf=clf.fit(X_train, y_train)
```

```python
print("Depth of the Decision Tree = ", clf.get_depth())
print("Total no. of leaf nodes in the Decision Tree = ", clf.get_n_leaves())
```

```
Depth of the Decision Tree =  7
Total no. of leaf nodes in the Decision Tree =  16
```

```python
#Train Error
y_train_predicted_labels = clf.predict(X_train)

#Test error
y_test_predicted_labels=clf.predict(X_test)

#Accuracy

print("\n Train Accuracy :", accuracy_score(y_train, y_train_predicted_labels))
print("\n Test Accuracy  :", accuracy_score(y_test, y_test_predicted_labels))
```

```
 Train Accuracy : 1.0

 Test Accuracy  : 0.9239766081871345
```

```python
# Generate DT Visulization
# Export the decision tree to DOT format
dot_data = tree.export_graphviz(clf, feature_names=feature_names,class_names=target_names,filled=True,out_file=None)

# Create a graph from DOT data
graph = graphviz.Source(dot_data)

# Display the graph directly in the notebook
display(graph)

# To save the visualization to a file
graph.render("default_params", format="png", cleanup=True)
```

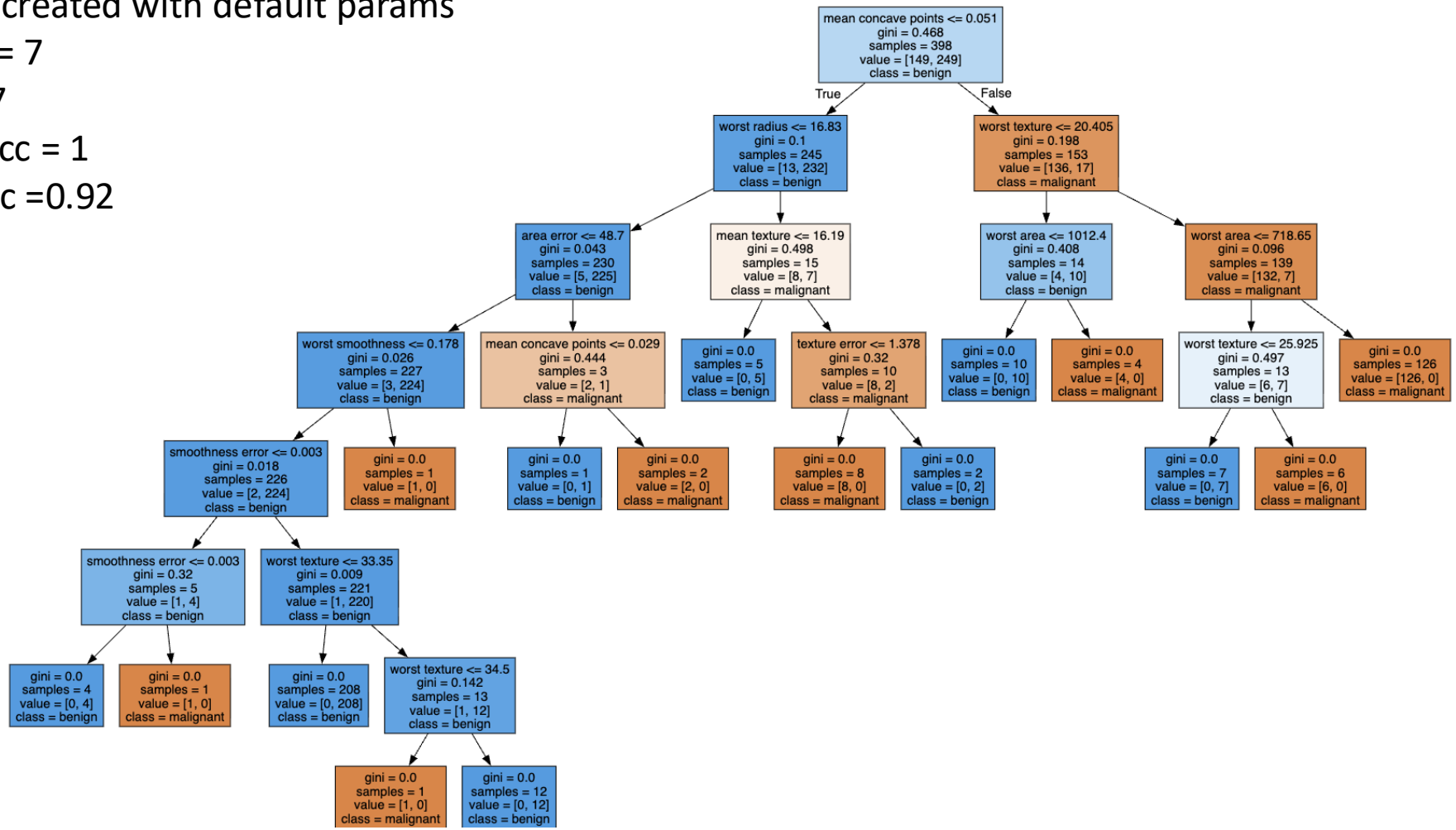
Model created with default params

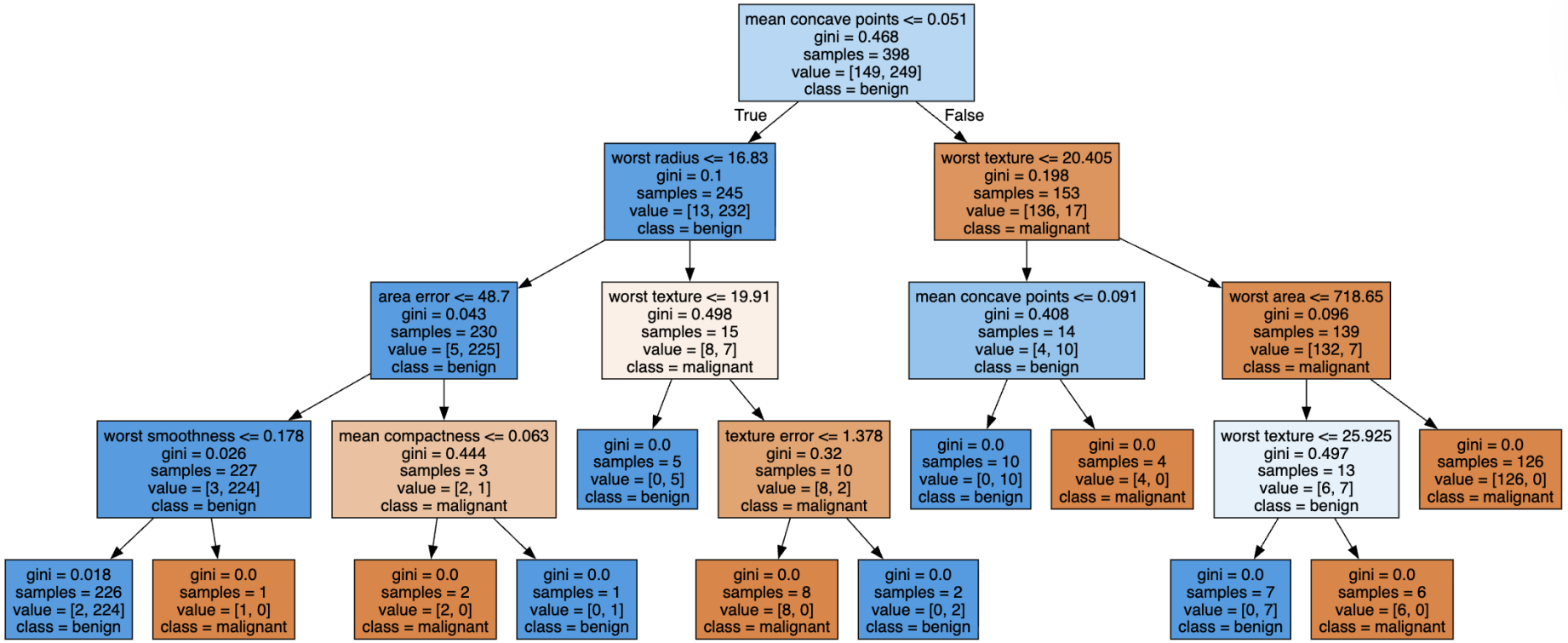
Depth = 7

LN = 17

Train Acc = 1

Test Acc =0.92



clf= tree.DecisionTreeClassifier(max_depth=4)

```
Depth of the Decision Tree =  4
Total no. of leaf nodes in the Decision Tree =  12

 Train Accuracy : 0.9949748743718593

 Test Accuracy  : 0.9473684210526315
```